



Product Requirements Document and 59-Hour Build Blueprint

Telegram-native, ERC-8004-identified anomaly intelligence for Mantle wallets - with every high-value alert and human-confirmed outcome proven on Mantle.

- AI Alpha & Data
- Telegram Bot
- ERC-8004
- Mantle-Native Proofs

PRODUCT LOCK

MantSent - Mantle Sentinel

BUILD BUDGET

Approx. 59-60 hours
4 full days + 9 evening sessions

PRIMARY TARGET

AI Alpha & Data track
Phase 2 submission

The submission to build

MantSent is a Telegram-native AI security analyst for Mantle wallets that detects abnormal outflows, explains the risk, records human-confirmed outcomes on Mantle, and builds a verifiable ERC-8004 identity and performance trail.

This document locks a single, winnable product for a constrained build window. It deliberately removes DeFi execution, Photon dependency, yield routing, Byreal integration, marketplaces and speculative complexity. The winning demonstration is a real Mantle transaction triggering a useful Telegram alert whose decision and verified outcome become publicly auditable.

TARGET TRACK

AI Alpha & Data

Mantle announced this track for smart-money tracking and on-chain anomaly detection bots delivered through Telegram and Discord. [S1]

CORE PROOF

Real Mantle Events

A controlled MNT outflow triggers an on-chain evidence-backed incident and human resolution trail.

TRUST LAYER

ERC-8004 Agent ID

The agent uses Mantle-deployed identity infrastructure and, if feasible, reputation feedback. [S2][S3]

Non-negotiable build thesis: do less, but make Mantle central and real. The product must show a Mantle wallet event, Mantle ERC-8004 identity, Mantle decision/outcome commitment and a public proof page.

Contents

01. Strategy, Mantle fit and prize target	03
02. PRD: problem, users and requirements	05
03. User experience and Telegram flow	08
04. Mantle-native system design	11
05. Contracts, ERC-8004 and data model	15
06. Detection logic, AI role and benchmarking	18
07. 59-hour execution plan and kill tests	21
08. Demo, submission, prizes and revenue	24
Appendices: interfaces, prompts and references	28

Why MantSent is the right Mantle build

MantSent is designed around the official Phase 2 language rather than forcing a generic agent into the ecosystem. The Mantle announcement identifies AI Alpha & Data as the track for smart-money tracking and on-chain anomaly detection bots via Telegram and Discord. [S1]

Track selection

Track	Alignment to MantSent	60-hour build risk	Decision
AI Alpha & Data	Telegram on-chain anomaly bot is the direct brief.	Low-medium	Primary target
AI x RWA	Requires credible USDY/mETH risk or yield logic.	High	Do not claim
Agentic Wallets & Economy	Announced as using Byreal Skills CLI.	High	Do not claim without Byreal
AI DevTools	Security adjacency, but less direct user value.	Medium	Not primary
Consumer / Viral	Could reward UX, not core fit.	Medium	Secondary narrative only

Hackathon defining features mapped to the product

Mantle requirement	MantSent implementation	Evidence judges can see
On-chain AI benchmarking	Every material alert and confirmed outcome is committed to a Mantle contract; incident detail is hash-linked off-chain.	Mantle explorer transactions + proof timeline.
ERC-8004 agent identity	Register one MantSent agent through the deployed Mantle Identity Registry; expose its agentId and agentURI.	Registry transaction + agent metadata card.
Radical transparency	Real Telegram alert, human classification and public incident proof page; include one false-positive or expected-transfer resolution.	Live demo or recorded real flow with auditable decisions.

Brand compatibility rule: every critical screen and transaction must visibly say Mantle, MNT, ERC-8004 and MantSent. Avoid a generic EVM app with a late Mantle deployment.

What is being removed

CUT FROM BUILD

- Photon / Spectrum dependency
- Byreal and DeFi execution
- USDY/mETH strategy proposals
- Wallet custody and approvals
- Marketplace, payments and billing

RETAINED AS CORE

- Telegram bot surface
- Real Mantle wallet monitoring
- ERC-8004 identity
- Human-resolved anomaly alerts
- Proof page and benchmark trail

The useful product underneath the prize submission

2.1 Problem statement

Small Web3 teams and individual treasury holders often monitor wallets manually. Large outflows, first-seen recipients or broken treasury policies may be noticed only after funds leave. Generic explorers show transactions; generic alert bots do not provide interpretable risk reasoning, policy-aware severity or a verifiable record of whether prior alerts were right.

Problem to solve: a Mantle wallet operator needs immediate, understandable, policy-aware anomaly alerts in Telegram and a tamper-evident record of the agent's decision accuracy.

2.2 Product objective

MantSent converts a plain-English treasury risk rule into structured monitoring logic, watches a Mantle wallet for controlled anomaly categories, delivers a high-signal Telegram alert with transaction evidence, captures human resolution, and records agent decisions/outcomes on Mantle against an ERC-8004 identity.

2.3 Target user

Persona	Pain	Why MantSent is useful
Protocol founder / operator	Cannot continuously monitor operational wallets.	Telegram alert for unusual transfers and proof record.
Small DAO multisig signer	Wants visibility before/after treasury movement.	Policy-based incident detection and shared trail.
Mantle power user	Wants fast context on risky wallet actions.	First-seen address + threshold alert in familiar chat.
Judge / ecosystem reviewer	Needs to verify AI behaviour, not trust a video claim.	ERC-8004 identity plus on-chain alert/outcome evidence.

2.4 Value proposition

Most alert bots can delete bad calls and show only successes. MantSent cannot hide its history: its material alert decisions and human-confirmed outcomes are anchored on Mantle.

2.5 Success metrics for the hackathon build

Metric	Pass criterion	Why it matters
End-to-end latency	Controlled MNT transfer produces Telegram alert in under 60 seconds.	Feels live in demo.
Mantle identity	One ERC-8004 agentId is visibly registered on Mantle.	Meets identity narrative.
Benchmark trail	Policy, alert and outcome each have Mantle proof records.	Meets on-chain benchmarking narrative.
Resolution	User labels alert Expected or Suspicious in Telegram.	Creates verifiable outcome signal.
Judge readability	Proof page explains the complete event in under 30 seconds.	Submission conversion.

Scope, user stories and acceptance criteria

2.6 Must-have scope

ID	Feature	Acceptance criterion
M1	Telegram bot onboarding	User can start bot, see product purpose and create one MantSent agent.
M2	ERC-8004 identity	Agent creation results in a real Mantle identity registry transaction or, if final registry constraints prevent testnet use, a documented verified fallback approved before submission.
M3	Watch wallet	User can add one Mantle address; backend persists address and active policy.
M4	Plain-English policy	User rule is parsed to structured threshold + recipient novelty criteria and confirmed back to user.
M5	Real event detector	Backend detects MNT native outflow above threshold from watched wallet.
M6	Novel-recipient classification	Recipient not previously seen in monitoring window increases severity deterministically.
M7	AI explanation	Bot sends concise reason, evidence transaction, policy violated and recommended next check.
M8	Human resolution	Inline buttons mark an event Expected or Suspicious and persist outcome.
M9	Mantle proof ledger	Policy commitment, alert commitment and outcome transaction are written on Mantle.
M10	Public passport	Public URL displays agent ID, policy, incident timeline, proof hashes and performance summary.

2.7 Stretch scope - only after M1-M10 pass

SAFE STRETCH

- ERC-20 Transfer event monitoring
- Real ERC-8004 reputation feedback from a separate verifier wallet
- Policy edit after Expected Transfer outcome
- Small historical scan for recent wallet transactions

DO NOT TOUCH IN THIS BUILD

- Autonomous asset movement
- Trade/yield execution
- Multi-wallet analytics platform
- Graph analytics / whale labels
- Payments, subscriptions or grants integration

2.8 User stories

- As a Mantle treasury operator, I can register a monitoring agent in Telegram so I do not need a new dashboard to receive protection.
- As a treasury operator, I can say "alert me for more than 10 MNT sent to a new address" and receive a confirmed structured policy.
- As a judge, I can see the agent's identity and a permanent record of its alert decisions and outcomes on Mantle.
- As a user, I can mark whether an alert was expected or suspicious so the system does not claim unverified accuracy.

Accuracy promise guardrail: MantSent is an anomaly signal and audit trail tool, not proof of theft and not a security guarantee. UI copy must use "suspicious" or "unexpected" until confirmed by the operator.

One polished conversational loop

Telegram is not a fallback UI here. It is a first-class channel explicitly aligned with the AI Alpha & Data track wording. The product should feel immediate, serious and low-noise.

3.1 Primary user journey



3.2 Bot commands

Command	Behaviour	Priority
<code>/start</code>	Explains purpose; provides Create Agent button.	Must
<code>/create</code>	Registers MantSent identity and creates passport URL.	Must
<code>/watch 0x...</code>	Sets one Mantle wallet to monitor.	Must
<code>/policy <text></code>	Converts plain-English rule to deterministic criteria.	Must
<code>/incidents</code>	Lists recent alerts and resolution state.	Should
<code>/proof</code>	Opens public agent passport.	Must

3.3 Golden-path conversation copy

```
USER /create

MANTSENT Your Mantle Sentinel is live.
Agent type: Treasury Anomaly Monitor
ERC-8004 Agent ID: #1024
Network: Mantle
Next: add a wallet with /watch 0x...

USER /watch 0xTreasuryWallet
MANTSENT Watching this Mantle wallet. Set a risk rule with /policy.

USER /policy Alert me if more than 10 MNT leaves this wallet,
        especially if the recipient is new.

MANTSENT Policy active on Mantle:
- Asset: MNT
- Trigger: outflow greater than 10 MNT
- Escalation: new recipient = Critical
Proof: PolicyCommitted transaction
```

3.4 The alert card

CRITICAL MANTLE TREASURY ALERT

Detected: 25 MNT outflow

Recipient: First-seen address

Policy trigger: amount exceeds 10 MNT and recipient is new

Why this matters: a large transfer to a first-seen recipient may be an unauthorised payout or compromised signer action.

Evidence: Mantle transaction 0x...

[Expected Transfer] [Suspicious Activity] [View Proof]

Judge-facing UX and brand rules

3.5 Agent Passport page

The public page is not a dashboard project. It is one vertically-scannable proof page designed for judges and potential customers.

HEADER

- MantSent wordmark
- ERC-8004 agentId
- Mantle chain / explorer link
- Monitoring status

POLICY CARD

- Plain-language policy
- Structured rule criteria
- Policy commitment transaction

INCIDENT TIMELINE

- Detected transaction
- Severity and explanation
- Human resolution
- Outcome proof

VERIFIED PERFORMANCE

- Alerts issued
- Confirmed suspicious
- Expected transfers
- Unresolved events

3.6 Mantle-themed presentation rules

Area	Use this	Avoid this
Language	"Mantle wallet", "MNT outflow", "proof on Mantle", "ERC-8004 identity"	Generic "EVM alert app" wording
Demo asset	Native MNT on Mantle for reliable real event	Fake numbers with no chain evidence
Explorer	Visible Mantle explorer transaction links	Hidden chain calls
Identity	Existing ERC-8004 Mantle registry if verified	Custom NFT presented as standard-compliant without proof
Design	Dark graphite + green accent; security-grade clean UI	Memecoin visuals or flashy trading UI

3.7 Transparency without overclaiming

DECISION

The agent classified a real transaction based on a user-defined policy.

OUTCOME

The human operator labelled whether the transaction was expected or suspicious.

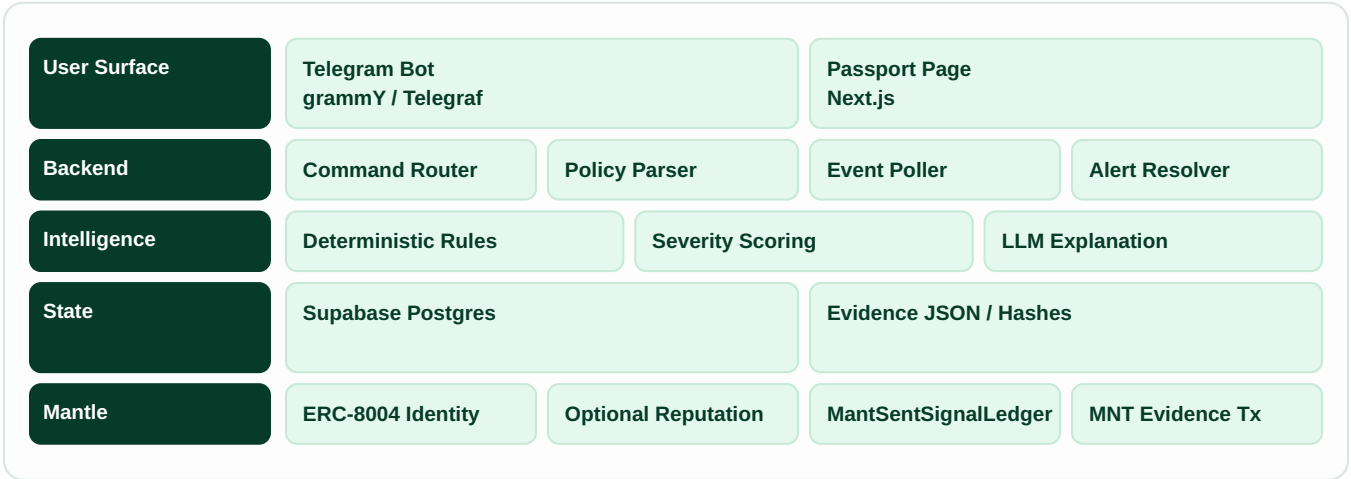
REPUTATION

Stats reflect labelled outcomes, not claimed fraud detection accuracy.

Winning narrative: MantSent is not claiming to stop all treasury attacks. It makes a narrow class of AI alerts verifiable: every material signal has public evidence and every resolved outcome stays attached to the agent identity.

Centralized agent runtime, decentralized benchmark

The runtime is intentionally simple: one TypeScript service hosts the bot, monitor and decision pipeline. Mantle provides identity and tamper-evident proof, not application orchestration.



4.1 Component responsibilities

Component	Responsibility	Failure-safe behaviour
Telegram adapter	Receive commands, send alerts and capture resolution buttons.	Never lose outcome action; retry transient send failure.
Agent service	Create local profile, register ERC-8004 identity and link passport.	If registry fails, surface failure - do not pretend identity exists.
Policy parser	Extract threshold and new-recipient condition from user text.	Require user confirmation before activation.
Mantle event poller	Check watched address for outbound MNT transaction activity.	Use checkpointed block cursor; idempotent event processing.
Anomaly engine	Apply deterministic policy and severity calculation.	No alert commitment until evidence tx is present.
LLM explainer	Convert deterministic reason into readable alert narrative.	Falls back to template if model unavailable.
Proof writer	Write policy, alert and outcome commitments to Mantle.	Telegram marks "proof pending" until confirmed.
Passport API	Present verified timeline and links.	Separates DB detail from on-chain proof status.

4.2 Why this is buildable

ONE CHAIN READ

Monitor native MNT transactions from one controlled treasury wallet.

ONE CUSTOM CONTRACT

Signal ledger emits policy, alert and outcome events only.

ONE LIVE DEMO

Send >10 MNT to first-seen recipient and resolve the alert.

ONE TRUST STORY

ERC-8004 identity plus Mantle-anchored decisions.

Core sequences

4.3 Create agent and policy

```
User -> Telegram Bot: /create
Telegram Bot -> Backend: createAgent(telegramUserId)
Backend -> ERC-8004 Identity Registry on Mantle: register(agentURI)
Identity Registry -> Backend: agentId + txHash
Backend -> DB: store agent / proof state
Backend -> Telegram Bot: Agent ID + proof page

User -> Telegram Bot: /policy Alert for >10 MNT to a new recipient
Backend -> LLM: parse policy candidate
Backend -> User: confirm structured rule
User -> Backend: confirm
Backend -> SignalLedger on Mantle: commitPolicy(agentId, policyHash)
Backend -> Telegram Bot: Policy active + Mantle proof
```

4.4 Detect, alert and resolve

```
Demo Treasury -> Mantle: transfer 25 MNT to new address
Event Poller -> Mantle RPC: detect transaction receipt / new block
Event Poller -> DB: load policy and recipient history
Anomaly Engine: amount > threshold AND first-seen recipient = CRITICAL
Proof Writer -> SignalLedger: commitAlert(alertHash, evidenceTxHash, severity)
LLM Explainer -> Telegram: human-readable critical alert

User -> Telegram: [Suspicious Activity]
Backend -> DB: persist labelled outcome
Proof Writer -> SignalLedger: recordOutcome(alertHash, SUSPICIOUS, feedbackHash)
Backend -> Passport Page: verified timeline updated
Backend -> Telegram: Outcome recorded on Mantle
```

4.5 Idempotency and reliability

Concern	Phase 1 approach
Polling same transaction twice	Unique constraint on chainId + evidenceTxHash + watched wallet.
Contract write pending	Status fields: queued, submitted, confirmed, failed; alert indicates proof status.
LLM fails	Use deterministic template based on severity and policy trigger.
Bot callback retried	Outcome buttons are idempotent; repeated clicks do not create duplicate proof writes.
RPC instability	Cache last confirmed block cursor; provider fallback only if needed after core works.

Chain integration that judges can verify

5.1 On-chain commitments

Proof type	What is written on Mantle	What remains off-chain
Agent identity	ERC-8004 Identity Registry agentId + agentURI pointer.	Human-readable agent metadata file.
Policy	Hash of structured policy and timestamp.	Raw Telegram text + parsed JSON.
Alert	Hash of alert JSON, monitored wallet, evidence transaction hash, severity.	Explanation text, policy details, bot message ID.
Outcome	Alert hash, outcome enum and feedback hash.	User action metadata and full resolution record.
Reputation (stretch)	ERC-8004 feedback from a valid separate evaluator identity/wallet if implementation permits.	Derived summary and interpretation.

5.2 ERC-8004 integration decision

ERC-8004 defines an Identity Registry based on ERC-721 agent identities and a Reputation Registry for feedback signals. The curated ERC-8004 contracts repository lists deployed Identity and Reputation registry addresses for Mantle Mainnet and Mantle Testnet. [S2][S3]

MANTLE MAINNET - VERIFY BEFORE USE IdentityRegistry 0x8004A169FB4a3325136EB29fA0ceB6D2e539a432 ReputationRegistry 0x8004BAa17C55a88189AE136b182e5fdA19dE9b63	MANTLE TESTNET - VERIFY BEFORE USE IdentityRegistry 0x8004A818BFB912233c491871b3d84c89A494BD9e ReputationRegistry 0x8004B663056A597Dffe9eCcC1965A193B7388713
---	---

Day 1 verification gate: confirm addresses and callable ABIs directly against Mantle explorer and the current hackathon rules. Confirm whether testnet transactions qualify for submission and the Finalist & Deployment award before relying on testnet-only proof.

5.3 MantSentSignalLedger - only custom contract

```
event PolicyCommitted(
    uint256 indexed agentId,
    bytes32 indexed policyHash,
    address indexed watchedWallet,
    uint256 timestamp
);

event AlertCommitted(
    uint256 indexed agentId,
    bytes32 indexed alertHash,
    address indexed watchedWallet,
    bytes32 evidenceTxHash,
    uint8 severity,
    uint256 timestamp
);

event OutcomeRecorded(
    uint256 indexed agentId,
    bytes32 indexed alertHash,
    uint8 outcome,
    bytes32 feedbackHash,
    uint256 timestamp
);
```

05 / ON-CHAIN DESIGN CONTINUED

Evidence, metadata and honest reputation

5.4 Agent metadata file

```
{
  "name": "MantSent - Mantle Sentinel",
  "description": "Telegram-native anomaly intelligence for Mantle wallets.",
  "capabilities": [
    "mantle-wallet-monitoring",
    "mnt-outflow-anomaly-detection",
    "telegram-alerting",
    "human-resolved-outcome-recording"
  ],
  "service": {
    "type": "telegram",
    "endpoint": "https://t.me/MantSentBot"
  },
  "proofPage": "https://.../agents/{agentId}",
  "network": "Mantle"
}
```

5.5 Alert evidence JSON

```
{
  "agentId": "1024",
  "policyHash": "0x...",
  "watchedWallet": "0x...",
  "evidenceTxHash": "0x...",
  "detected": {
    "asset": "MNT",
    "outflowAmount": "25",
    "threshold": "10",
    "recipientFirstSeen": true
  },
  "severity": "CRITICAL",
  "reasonCodes": ["THRESHOLD_BREACH", "NEW_RECIPIENT"],
  "explanationModel": "template-plus-llm",
  "timestamp": "2026-06-..."
}
```

5.6 Reputation model: narrow and honest

Phase 1 should display a **verified outcome summary**, not claim a universal security score. ERC-8004 reputation is valuable only if feedback originates from an eligible external evaluator rather than self-awarded evidence.

Displayed metric	Definition	Phase 1 source
Alerts raised	Total committed alerts for this agent.	SignalLedger events
Resolved events	Alerts receiving Expected or Suspicious outcome.	OutcomeRecorded events
Confirmed suspicious	Resolved events marked Suspicious by operator.	OutcomeRecorded
Expected transfers	Resolved events marked Expected.	OutcomeRecorded
Transparency coverage	Resolved alerts with both alert and outcome proof recorded.	Derived from Mantle events

Stretch reputation signal: use a separate verifier wallet or judge/demo evaluator to submit one ERC-8004 reputation feedback signal tagged `treasury-anomaly` and `human-confirmed`, only after validating the registry rules.

06 / APPLICATION DESIGN

Database, endpoints and detection logic

6.1 Minimal database schema

```
agents
  id, telegram_user_id, erc8004_agent_id, agent_uri,
  identity_tx_hash, passport_slug, created_at

watched_wallets
  id, agent_id, address, chain_id, cursor_block, created_at

policies
  id, agent_id, wallet_id, raw_text, structured_json,
  policy_hash, proof_tx_hash, active, created_at

alerts
  id, agent_id, wallet_id, evidence_tx_hash, recipient,
  outflow_amount, severity, reason_codes, alert_hash,
  proof_tx_hash, status, created_at

outcomes
  id, alert_id, user_label, feedback_hash,
  proof_tx_hash, created_at
```

6.2 Internal API surface

Endpoint / handler	Use	Exposed?
POST /webhooks/telegram	Bot inbound messages and button callbacks.	Public webhook
POST /agents	Create agent and start registry workflow.	Bot only
POST /agents/:id/watch	Persist watched wallet.	Bot only
POST /agents/:id/policy	Parse, confirm and commit policy.	Bot only
POST /alerts/:id/resolve	Write human outcome.	Bot only
GET /passport/:slug	Public verified performance page.	Public

6.3 Deterministic anomaly rule

```
Input:
  outbound MNT transaction from watchedWallet
  policy.thresholdMnt
  policy.escalateNewRecipient
  previouslySeenRecipients[]

Evaluation:
  thresholdBreachd = amountMnt > policy.thresholdMnt
  newRecipient = recipient not in previouslySeenRecipients

  if thresholdBreachd and newRecipient:
    severity = CRITICAL
    reasonCodes = [THRESHOLD_BREACH, NEW_RECIPIENT]
  else if thresholdBreachd:
    severity = HIGH
    reasonCodes = [THRESHOLD_BREACH]
  else:
    ignore in Phase 1
```

6.4 Role of AI vs code

Task	LLM?	Reason
Extract policy threshold from natural language	Yes, with confirmation	Convenient UX; user confirms deterministic representation.
Find transaction / amount / recipient	No	Chain evidence must be deterministic.
Classify threshold breach and novelty	No	Auditable and reproducible.
Write incident explanation	Yes, fallback template	Makes alerts readable without affecting truth.
Commit hash / record outcome	No	Must be exact and idempotent.

The 59-hour build plan

Codex is used aggressively for scaffolding, ABI typing, tests, component work and refactors. It does not change the external-dependency gates: ERC-8004 transactions, Mantle monitoring, Telegram callbacks and final deployment must be proven early.

7.1 Four full working days

Day	Hours	Deliverable	Exit condition
Full Day 1	8	Telegram reply; Mantle RPC write; ERC-8004 identity attempt; SignalLedger deployed; proof visible on simple page.	Kill test: Telegram -> Mantle write -> visible proof works.
Full Day 2	8	Watch wallet; policy parser; MNT event poller; controlled outflow triggers alert.	Real transfer -> Telegram alert works.
Full Day 3	8	Expected/Suspicious buttons; OutcomeRecorded; passport timeline; performance summary.	Full benchmark loop works twice reliably.
Full Day 4	8	Deployment, UI polish, reset script, README, architecture visual, demo video, submission.	Recorded submission flow and all proof links validated.

7.2 Nine evening sessions

Evenings 1-2 6 hours	Persistence and onboarding: Supabase schema, bot commands, agent create UI, policy confirmation.
Evenings 3-4 6 hours	Outcomes: callbacks, proof-writing retries, duplicate protection, success/error messaging.
Evenings 5-6 6 hours	Quality: incident explanation, proof page styling, explorer links, optional ERC-8004 feedback check.
Evenings 7-9 9 hours	Buffer: RPC failures, Telegram deployment, demo rehearsal, video editing, final submission fixes.

TOTAL BUILD 59 hrs Hard budget. No extra scope.	EXTERNAL GATES Day 1 Chain writes and identity tested first.	GOLDEN DEMO One incident Perfectly executed, real and provable.
--	---	--

Backlog, kill rules and Codex workflow

7.3 Ordered backlog

Priority	Ticket	Definition of done
P0	Repo setup and deployment skeleton	Frontend/backend online; secrets managed.
P0	Telegram webhook + response	/start returns stable response in hosted environment.
P0	Mantle client + wallet	Can read network and submit test transaction.
P0	ERC-8004 registry integration	Registered agent or documented blocker surfaced immediately.
P0	SignalLedger contract	Deployed and event readable from explorer/app.
P1	Policy and watcher state	One active wallet + rule persisted.
P1	MNT transaction detector	Detect controlled outflow exactly once.
P1	Alert message + explanation	Critical alert visible in Telegram with proof.
P1	Resolution + outcome proof	Button writes outcome; visible on page.
P1	Passport page	Judge understands full story in 30 seconds.
P2	Stretch ERC-8004 reputation	Only if all above complete and rule-compliant.

7.4 Kill rules

End of Full Day 1: if Telegram -> Mantle proof write -> visible explorer/page proof does not work, stop adding features. Spend at most one buffer evening rescuing the dependency; otherwise abandon the prize build rather than losing the full 59-hour budget.

Problem	Time cap	Fallback
ERC-8004 registry call cannot be completed	3 hours	Verify ABI/network/address; seek hack support. Do not label a custom NFT as compliant.
Telegram hosting callback issues	2 hours	Switch bot framework or polling during demo only; retain Telegram UI.
RPC event monitoring is flaky	3 hours	Poll receipts/blocks from controlled transaction rather than building indexing infra.
LLM explanations delay flow	30 minutes	Use deterministic explanation templates.
ERC-8004 reputation uncertain	1 hour	Cut stretch; display SignalLedger verified outcomes only.

7.5 Codex operating rule

Use Codex for fast implementation, but accept no generated module without a narrow verification step: run tests, check transaction receipt, reproduce bot callback, and inspect proof page. This is especially important for ABI integration, hashes, idempotency and chain/network configuration.

Demo, submission and prize positioning

8.1 Three-minute demo script

Time	What happens on screen	Message to judge
0:00-0:25	Problem statement + MantSent logo.	Treasuries need real-time, accountable anomaly intelligence.
0:25-0:55	Create agent in Telegram; open ERC-8004 identity transaction.	This agent has a real Mantle identity.
0:55-1:15	Watch wallet and set rule: >10 MNT to new recipient.	The user controls what risk means.
1:15-1:45	Trigger 25 MNT transfer; alert appears in Telegram.	AI is responding to a real Mantle event.
1:45-2:05	Mark Suspicious; OutcomeRecorded transaction shown.	Outcomes are human-confirmed and immutable.
2:05-2:45	Open passport timeline and proof links.	Agent performance is auditable, not self-reported.
2:45-3:00	Close on track fit.	MantSent makes Mantle the benchmark layer for AI security alerts.

8.2 Submission one-liner

MantSent is an ERC-8004-identified Telegram anomaly agent that monitors Mantle wallets, detects high-risk MNT outflows, captures human verification, and permanently benchmarks every alert outcome on Mantle.

8.3 Prize targeting

The prize figures below are taken from the Phase 2 prize structure screenshot supplied during planning and must be confirmed in the live portal before final submission. [S5]

Target	Published/prior supplied amount	MantSent case	Estimated odds if shipped cleanly
AI Alpha & Data track first	\$8,500	Exact Telegram anomaly-bot fit plus on-chain benchmark.	5-12%
Finalist & Deployment	\$1,000 each; 20 awards shown	Real deployment and demonstrable end-to-end proof.	20-35%
Best UI/UX	\$3,000	Polished Telegram + passport flow.	2-5%
Grand Champion	\$9,000	Harder: requires broad completion and standout story.	<2%
Any cash prize	-	Primarily finalist or track win.	22-38%

Odds are judgement estimates, not promises. They depend heavily on final submission count, rule compliance, demonstration reliability and how many teams build direct track-aligned products.

09 / AFTER THE HACK

Revenue path and ecosystem continuation

MantSent should not be positioned as a trading alpha product. Its credible post-hack wedge is treasury incident monitoring for Mantle-native teams and high-value wallets.

9.1 Monetizable version

Plan	Customer	Possible pricing	Product delivered
Free	Individual builder	\$0	One wallet, basic threshold rule, limited alert history.
Pro	Founder / active wallet holder	\$9-\$29 monthly	Multiple alert policies, historical incidents, faster alerts.
Team	Small protocol / multisig team	\$49-\$199 monthly	Multiple wallets, Telegram group workflow, incident exports.
Treasury	DAO / protocol ops	\$199-\$499 monthly	Role controls, webhook integrations and policy dashboards.

9.2 Realistic six-month odds after a polished submission

Outcome	Estimated odds	What improves it
10-30 beta users	35-55%	Public launch, direct Mantle community outreach, free onboarding.
First paid user	12-25%	Real alert reliability, team workflows, strong founder conversations.
\$500 MRR	5-12%	At least 3-5 paying small teams.
Grant/ecosystem support	8-18%	Strong deployment, usage metrics and ecosystem fit.

9.3 Arc/Circle continuation without distracting the submission

After the Mantle submission, the monitoring-and-proof core may evolve into a USDC operations or payment-control agent for Circle Arc grants. That is a separate post-hack fork: MantSent proves anomaly intelligence on Mantle; an Arc version would prove approved USDC settlement workflows. Do not integrate Arc during the Mantle deadline.

Business discipline: the hackathon code becomes portfolio proof and a possible monitoring MVP; revenue requires user interviews and actual wallet operators who value alerts, not only a prize narrative.

Risks, submission checklist and definition of done

10.1 Risk register

Risk	Impact	Mitigation
ERC-8004 address/ABI or network mismatch	Loss of defining feature	Verify contract, ABI and explorer transaction on Day 1; consult organisers if ambiguous.
Testnet does not qualify for deployment prize	Prize eligibility reduction	Verify rules early; prepare low-cost mainnet proof transaction only if allowed/needed.
Telegram bot is unstable during demo	Visible demo failure	Host early, rehearse reset path, retain pre-recorded demo as fallback.
Alert appears generic / threshold-only	Weak AI narrative	Show policy extraction, contextual explanation and immutable outcomes.
Claims of reputation overreach	Trust/judge backlash	Call it verified outcomes; use ERC-8004 reputation only under valid feedback rules.
Scope creep	Missed submission	No execution, no RWA, no Photon, no extra chains before submission.

10.2 Submission checklist

- ☐ Public Telegram bot responds and core golden path works from fresh state.
- ☐ Agent ERC-8004 identity registration proof is publicly accessible on Mantle.
- ☐ MantSentSignalLedger deployed and contract verified if time permits.
- ☐ Controlled MNT outflow evidence transaction is publicly accessible.
- ☐ AlertCommitted and OutcomeRecorded transactions are linked from passport.
- ☐ Policy text, structured rule and hash commitment are readable.
- ☐ README states AI role versus deterministic rule role clearly.
- ☐ README states track: AI Alpha & Data, and explains direct fit.
- ☐ Three-minute demo video is recorded with real proof links.
- ☐ All prize and eligibility claims confirmed against current portal rules.

10.3 Definition of a winning-quality Phase 1 build

A judge can create or observe an ERC-8004 MantSent agent, watch a real MNT anomaly trigger a clear Telegram alert, label the outcome, and verify the complete decision trail on Mantle in under three minutes.

Interfaces and prompt contracts

A.1 Structured policy schema

```
type Policy = {
  asset: "MNT";
  eventType: "OUTFLOW";
  thresholdAmount: string;           // decimal-safe string
  escalateNewRecipient: boolean;
  severityForThreshold: "HIGH";
  severityForThresholdAndNewRecipient: "CRITICAL";
  userConfirmed: boolean;
};
```

A.2 Alert and outcome schema

```
type Alert = {
  agentId: string;
  watchedWallet: `0x${string}`;
  evidenceTxHash: `0x${string}`;
  amountMnt: string;
  recipient: `0x${string}`;
  firstSeenRecipient: boolean;
  severity: "HIGH" | "CRITICAL";
  reasonCodes: ("THRESHOLD_BREACH" | "NEW_RECIPIENT")[];
  alertHash: `0x${string}`;
  proofTxHash?: `0x${string}`;
};

type Outcome = {
  alertId: string;
  label: "EXPECTED" | "SUSPICIOUS";
  feedbackHash: `0x${string}`;
  proofTxHash?: `0x${string}`;
};
```

A.3 LLM policy extraction prompt contract

```
SYSTEM:
Extract only a MantSent MNT outflow monitoring policy.
Return JSON matching the provided schema. Do not infer actions,
execute transactions, or label a wallet as compromised.
If amount or recipient condition is unclear, return requiresClarification=true.

USER:
{telegram policy text}

VALIDATOR:
Reject any output not limited to MNT + OUTFLOW + threshold + new recipient flag.
Ask user to confirm before committing policy on Mantle.
```

A.4 LLM incident explanation prompt contract

```
SYSTEM:
Write a maximum 70-word treasury anomaly explanation.
Use only supplied evidence. Say "may indicate" and never assert theft.
Include the policy trigger and recommend checking signer activity.
Do not provide trading or investment advice.

INPUT:
{policy, amount, recipientFirstSeen, evidenceTxHash, severity}
```

APPENDIX B / SOURCES AND VALIDATION

Reference sources and verification gates

B.1 Sources used

[S1] Mantle / PRNewswire release distributed through Chainwire, "Mantle Launches Turing Test Hackathon 2026 backed by Tencent Cloud, Bybit, Byreal and BGA," 23 April 2026. Used for Phase 2 track definitions, \$100,000 pool, on-chain benchmarking, ERC-8004 requirement and transparency framing. <https://chainwire.org/2026/04/23/mantle-launches-turing-test-hackathon-2026-backed-by-tencent-cloud-bybit-byreal-and-bga/>

[S2] ERC-8004 team, *erc-8004/erc-8004-contracts* repository, README contract address list and registry architecture, accessed 31 May 2026. Used for Mantle Mainnet/Testnet IdentityRegistry and ReputationRegistry addresses and registry behaviour. <https://github.com/erc-8004/erc-8004-contracts>

[S3] Ethereum Improvement Proposal, *ERC-8004: Trustless Agents*, accessed 31 May 2026. Used for the Identity Registry / Reputation Registry model and on-chain/off-chain evidence split. <https://eips.ethereum.org/EIPS/eip-8004>

[S4] Mantle announcement, "Mantle Unlocks Autonomous Economy with ERC-8004 Deployment," 16 February 2026. Used to validate that Mantle announced official ERC-8004 deployment on mainnet. <https://www.prnewswire.com/in/news-releases/mantle-unlocks-autonomous-economy-with-erc-8004-deployment-302688553.html>

[S5] Phase 2 prize structure images supplied by builder in the planning conversation, May 2026. Used only for planning estimates; confirm prize categories and eligibility against the current hackathon submission portal before submission.

B.2 Mandatory verification before coding beyond the kill test

Item	Why validation is required	When
Submission deadline and allowed deployment network	Testnet/mainnet acceptance affects eligibility and demo plan.	Before Full Day 1 ends
ERC-8004 registry addresses and ABI on Mantle	Address/ABI drift invalidates identity claim.	During Full Day 1
Reputation Registry feedback eligibility	Self-feedback or invalid feedback could weaken trust narrative.	Only before stretch feature
Phase 2 prize structure and track submission rules	Prize figures may change or require specific deployment evidence.	Before final README/ video
Use of Mantle brand assets	Avoid unapproved marks or incorrect ecosystem representations.	Before public submission

Prepared as a focused internal PRD and implementation blueprint - 31 May 2026.